





RESTful APIs with Python

A DevCouRSE Workshop

Tim Meehan, Simon Stone

Research Computing @ ITC, Dartmouth College



See APIs in action

<https://dartgo.org/devcourse-api-demo>





What is an API?

👉 API stands for Application Programming Interface

What is an interface? (in software design)

- 🧩 A piece of software can be thought of in two parts:
 1. How it does what it does (the *implementation*)
 2. What the user of the software needs to put in and what they can get out (the *interface*)
- 🙌 The implementation rarely matters to a user
- 💯 The interface is all we need to know to use the software
- 👉 The implementation may change, as long as the interface stays the same
- 🤝 The interface is a contract between the program and the user

What is an interface? (in software design)

Example:

Description of the interface

Notification of a change
in the “contract”

Interface

```
print(*objects, sep=' ', end='\n', file=None, flush=False)
```

Print *objects* to the text stream *file*, separated by *sep* and followed by *end*. *sep*, *end*, *file*, and *flush*, if present, must be given as keyword arguments.

All non-keyword arguments are converted to strings like [str\(\)](#) does and written to the stream, separated by *sep* and followed by *end*. Both *sep* and *end* must be strings; they can also be None, which means to use the default values. If no *objects* are given, [print\(\)](#) will just write *end*.

The *file* argument must be an object with a `write(string)` method; if it is not present or None, [sys.stdout](#) will be used. Since printed arguments are converted to text strings, [print\(\)](#) cannot be used with binary mode file objects. For these, use `file.write(...)` instead.

Output buffering is usually determined by *file*. However, if *flush* is true, the stream is forcibly flushed.

→ *Changed in version 3.3:* Added the *flush* keyword argument.

No implementation
details!

Source: <https://docs.python.org/3/library/functions.html#print>

What is an API?

- 👉 API stands for Application Programming Interface
- 🧱 It allows us to use someone else’s “code blocks” to build applications/programs/scripts
- 🤖 A good API hides implementation details
- 📖 Specified by the required inputs and the promised outputs (the “contract”)
- 🌐 Commonly refers to web APIs in particular

Why (web) APIs?

-  Separation of concerns: Decouple logic (backend) from presentation (frontend)
-  Context independence: Build one API and use it in different contexts
-  Interoperability: Allow different services/stacks to talk to each other
-  Leverage ecosystems: Integrate third-party services
-  Scalability: Update or scale backend independently of frontend

RESTful APIs

REpresentational State Transfer (REST)

A way to structure network-based applications

An application is *RESTful*, if:

- **Resources** are being transferred (e.g., a web page, images, list of names, sensor data, ...)
- The user interacts with the application using a **limited set of actions**
- The application is **stateless**



RESTful APIs

REpresentational State Transfer (REST)

A way to structure network-based applications

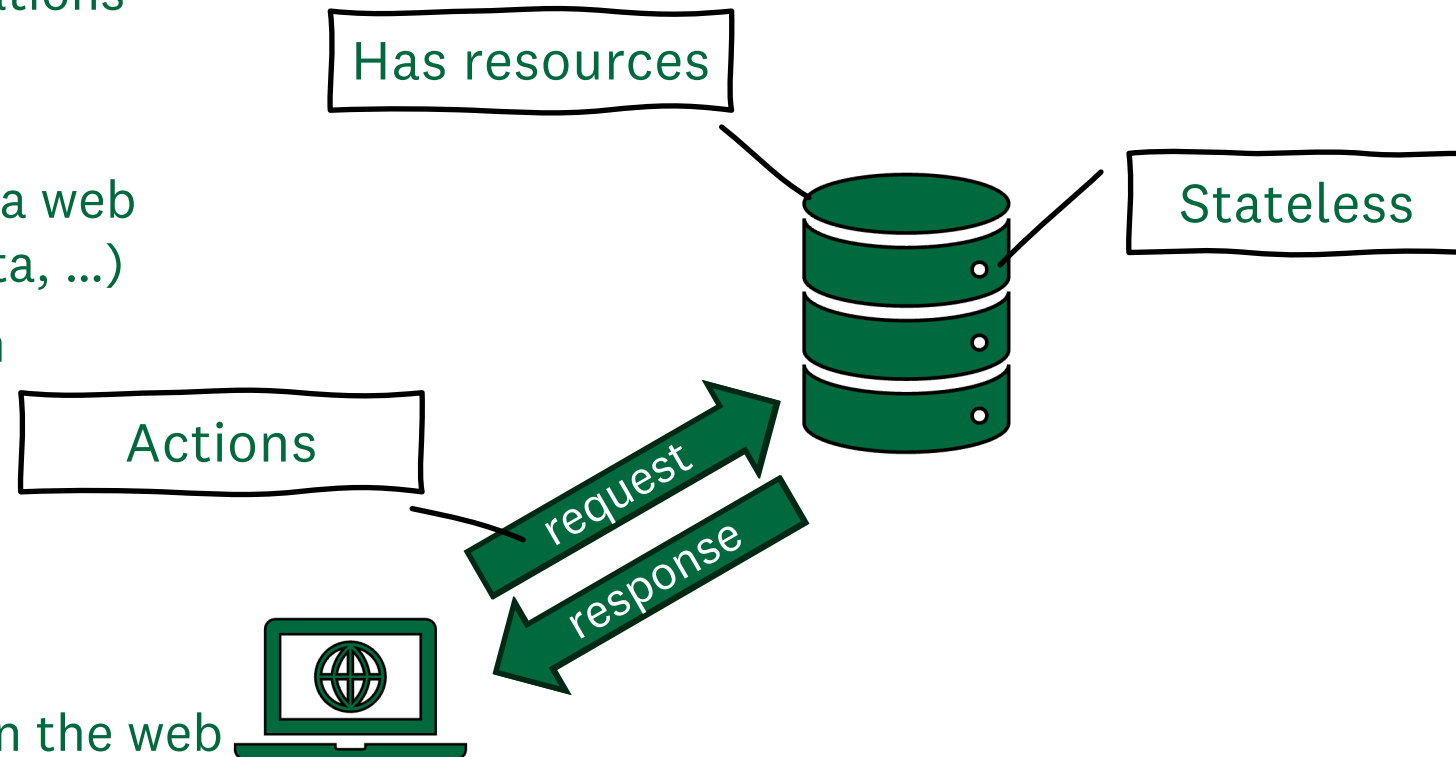
An application is *RESTful*, if:

- **Resources** are being transferred (e.g., a web page, images, list of names, sensor data, ...)
- The user interacts with the application using a **limited set of actions**
- The application is **stateless**

HyperText Transfer Protocol (HTTP)

The protocol governing communication on the web

👍 A (mostly) RESTful application





REST Actions <-> HTTP Methods

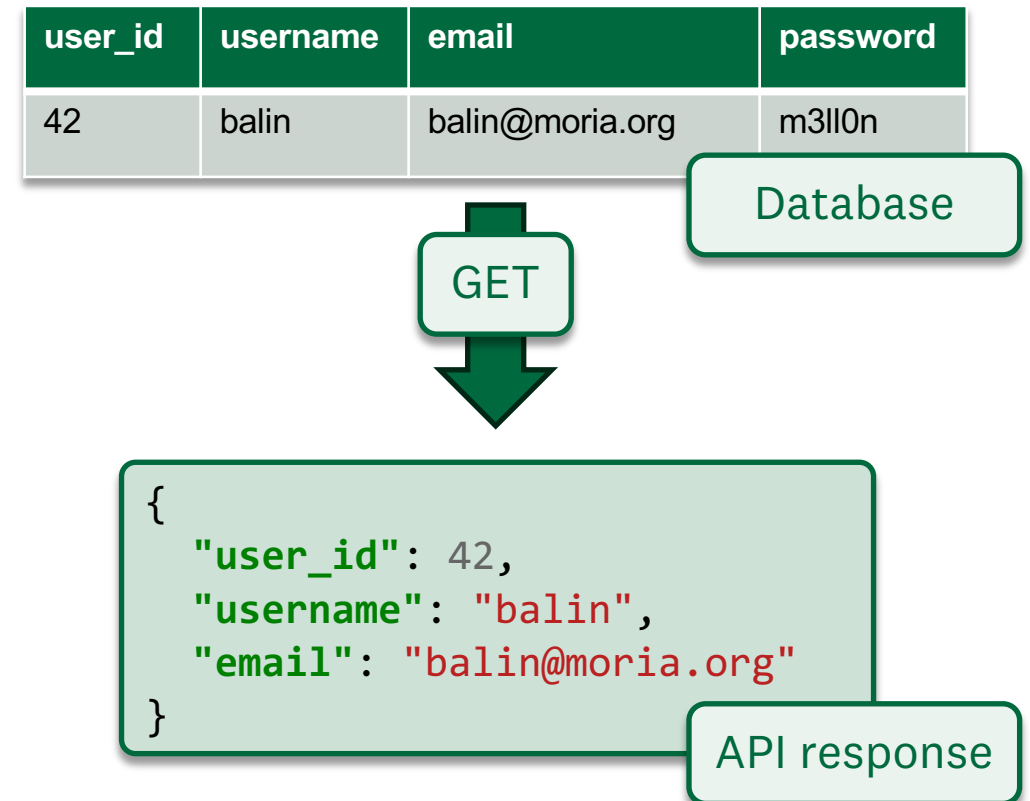
REST Action	HTTP Method	Example Endpoint	Purpose
Create	POST	/users	Create a new resource
Read	GET	/users/123	Retrieve specific resource
Update	PUT	/users/123	Update specific resource
Delete	DELETE	/users/123	Remove specific resource

Verbs

Nouns

REST APIs and Databases

- 📁 Resources are (typically) stored in SQL database tables
- 👉 Database tables (“models”) can be different from what the API returns (“schemas”)



A word on “Statelessness”

 Applications using RESTful APIs *can* have mutable state

 Just the *communication* is stateless

 Example: Pagination

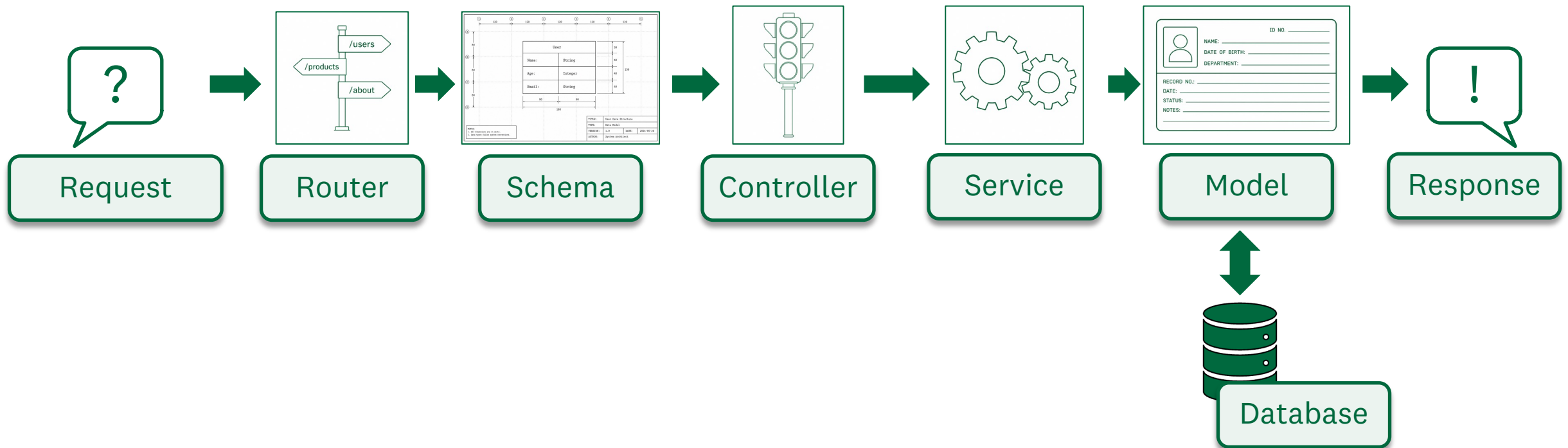
Non-RESTful: “Give me the next page of results.”

RESTful: “Give me page 2 of the results for User 123.”

 Each request must be self-contained

Each response must contain all information to continue (e.g., number of pages)

Life-cycle of a typical backend API request



Implementing RESTful APIs in Python

Today's stack:

- **FastAPI:** Handling routers, controllers, HTTP request mapping
- **SQLModel:** Handling schemas, models, and database I/O
- **Plain Python:** Implementation of services
- **SQLite:** database engine





Let's get into it!





Thank you